ARGUS in One Pass

What each part does, and what runs it

ARGUS in one pass

What each part does, why it is built that way, and the command that runs it. Commands are labelled **PI** or **LAPTOP**.

The shape of it

```
camera ──► motion ──► YOLO ──► tracking ──► events ──► federated head
                                              │              │
                                              ├─ still (jpg) ├─ "routine here" / "unusual here"
                                              ├─ clip (mp4)  │
                                              └─ database    └─ round ──► gate ──► live model
```

A camera node watches a scene, decides what is in frame, keeps only what matters, and learns what is normal **for that site**. Weights are shared between nodes; footage never is.

1 · Capture

A background worker owns the camera and runs capture and detection on separate threads. Detection does not depend on anyone watching the video (FR-19), and when the detector is slower than the camera it works on the newest frame rather than queueing stale ones.

A video file can stand in for the camera, which makes runs repeatable:

PI `./scripts/run_node.sh --video media/vtest.avi`

2 · Motion

Frame differencing on a 320 px copy, reusing the previous blur. At 1080p this went from 16.7 ms to 2.6 ms per frame.

It also acts as a gate: by default YOLO only runs when something moved. Cheap, but a subject that holds still is never classified.

PI `./scripts/run_node.sh --every-frame` runs YOLO on every frame. `motion_rate` in `/api/detection/status` shows how often the gate opens.

3 · Detection

YOLO, filtered to the four ARGUS classes before NMS:

Class	COCO detections
person	person
vehicle	bicycle, car, motorcycle, bus, train, truck
animal	bird, cat, dog, horse, sheep, cow
package	backpack, handbag, suitcase

COCO has no parcel class, so a cardboard box detects as nothing.

Backends, in the order they are preferred: TensorRT → Hailo → an optimised CPU export (NCNN on the Pi, OpenVINO on x86) → plain PyTorch. Exports have a fixed input size, which overrides `OBJECT_IMGSZ` — asking a 640 export for 416 returns zero detections, very fast.

PI `python scripts/export_cpu_models.py --format ncnn`

4 · Tracking and events

Detections are associated across frames by overlap, with a centre-distance fallback so a fast-moving car is not split into fragments. **One event per track, not per frame** — a person crossing the frame is one row, not thirty. Anything under three frames is discarded as flicker.

Each event stores a still taken at the subject's largest, because a confidence number is not reviewable and by the time a track closes the subject has usually left. Clips carry pre- and post-roll. Sustained motion with no object becomes its own row, cut into 60-second chunks so a scene that never goes still still gets recorded.

Kind	Where
Event + features	`DATA_DIR/training/samples.jsonl`
Still	`DATA_DIR/snapshots/`
Clip	`MEDIA_DIR/events/`
Rows	SQLite `events` table, with `media_path`

5 · The federated head

A ~205K parameter model that does **not** classify pixels. YOLO already says what a thing is; the head says whether it is normal **here**: class, size, position, speed, dwell, time of day. A van at 14:00 is routine; the same van at 03:00 is not. That judgement is site-specific, which is what makes it worth federating.

Labels: `routine_person` · `routine_vehicle` · `routine_animal` · `routine_package` · `anomaly`. The dashboard shows `anomaly` in red as "Unusual for this site". No faces are involved — face recognition was deprecated on 2026-09-14 after measuring it at 12–27× the frame cost.

Labels come from a person, through Confirm/Correct in the dashboard. The simulator derives labels from the same rule that generates its data, which measures throughput and proves nothing about accuracy.

6 · A round, and the gate

The node trains on its own samples, sends **weights only**, and the server averages them. The returned model is scored against this node's held-out samples and replaces the live one **only if balanced accuracy does not regress**. Accepted weights persist across restarts.

A rejection is the system working: an average that suits other sites can hurt this one.

PI `./scripts/fl_trial.sh` runs one by hand. **LAPTOP** `.\scripts\run_central_server.ps1` is the other half: a Flower server that averages what the cameras send, checkpoints every version of the combined model, and exposes a REST API and Prometheus metrics. It serves sessions back to back and resumes from its last checkpoint, because a camera on a fortnightly schedule connects days after the previous session ended.

With `FL_ENABLED=true`, `FL_SERVER_URL` and `FL_CENTRAL_API` set, the node runs rounds on its own schedule — every 14 days at 2am, only while idle, catching up if the Pi was off across the due date. The scheduled round and the manual one call the same code (`src/federated/rounds.py`), so the demo cannot drift from what runs unattended.

Training runs in its own process on one core, so detection continues — on x86 the measured cost was 0.1% of frame rate.

Recordings are training data too: `scripts/ingest_video.py` replays a clip through the detector and labels everything it produces in one pass, using the clip's own frame rate so speed and dwell come out right.

7 · Two dashboards

- **Built into the node**, `http://<pi>:8000/dashboard`. Same origin as the API: no CORS, no install. Live feed, FPS every second, events with stills.
- **The product UI**, `adam9798/argus-dashboard`, on the laptop. Login, event review, Confirm/Correct labelling. **LAPTOP** `.\scripts\run_dashboard.ps1 -Node 10.0.0.140`

Settings worth knowing

Variable	Default	Effect
`MOTION_GATE`	`true`	`false` classifies every frame
`OBJECT_MODEL`	`yolov8n`	`yolov8s`, `yolov8m` — needs a matching export
`OBJECT_BACKEND`	`auto`	`pt` forces plain PyTorch
`CAMERA_SOURCE`	camera	a video file, for repeatable runs
`ARGUS_DATA_DIR`	`data`	moves DB, samples and model together
`RECORD_EVENTS`	`true`	clip recording
`CAMERA_SWAP_RB`	`false`	only if people look blue
`THREAT_ENABLED`	`true`	~100 ms/frame on the Pi's CPU

Measured, and not

Measured on x86 pinned to four cores, on a pedestrian video: dropping face recognition took 0.86 → 23.4 FPS; the smaller motion frame took 17.8 → 22.9 FPS at 1080p; YOLO at 416 instead of 640 gives ~60% more FPS for ~5% recall.

On the Pi, only two things are measured so far: the Hailo runs YOLOv8m at **76 FPS** via `hailortcli`, and the NCNN export detects correctly. **ARGUS itself does not use the Hailo yet** — that needs a `.hef` compiled on an x86_64 Linux machine. Until then the Pi runs NCNN on CPU, and no end-to-end Pi frame rate has been recorded.